

HamraGraph Application Documentation

1. Overview

HamraGraph is a Java-based application modeling a portion of Hamra, Beirut, as a weighted graph. Each vertex represents a location such as an intersection or landmark. Edges represent walkable paths, with distances computed using scaled Manhattan distance. The program supports searching, listing by type, nearest neighbor detection, and shortest path computation using Dijkstra's algorithm.

2. Core Components

- ConnectionPoint class storing vertex data.
- Vertex array storing up to 200 locations.
- Adjacency matrix storing distances.
- Full graph construction (addAllVertices/addAllEdges).
- Shortest path calculation using Dijkstra.
- Interactive text-based interface.

2.1 ConnectionPoint Class (Code Snippet)

Fields:

public String name;

public String kind;

public double x,

```
public static class ConnectionPoint {  
    public String name;  
    public String kind;  
    public double x, y;  
    public ConnectionPoint(String name, String kind, double x, double y) {  
        this.name = name;  
        this.kind = kind;  
        this.x = x;  
        this.y = y;  
    }  
    @Override  
    public String toString() {  
        return name + " (" + x + ", " + y + ") - " + kind;  
    }  
}
```

3. Edge and Vertex Construction (Code Snippet)

Edges connect landmarks and intersections. The distance between two points is computed using Manhattan distance: $(|dx| + |dy|) \times \text{SCALE}$.

```
public static void addVertex(String name, String kind, double x, double y) {
    if (vertexCount + 1 < MAX) V[++vertexCount] = new ConnectionPoint(name, kind, x, y);
}

public static void addEdge(int i, int j) {
    if (i <= 0 || j <= 0 || i > vertexCount || j > vertexCount) return;
    if (V[i] == null || V[j] == null) return;
    double dx = V[i].x - V[j].x;
    double dy = V[i].y - V[j].y;
    double dist = (Math.abs(dx) + Math.abs(dy)) * SCALE;
    E[i][j] = E[j][i] = dist;
}
```

4. Pathfinding Using Dijkstra (Code Snippet)

HamraGraph uses Dijkstra's algorithm to find the shortest path between two locations. A priority queue ensures efficient vertex selection

```
public static List<Integer> dijkstraPath(int src, int dest) {
    int n = vertexCount;
    double INF = Double.MAX_VALUE / 4;
    double[] dist = new double[n + 1];
    boolean[] used = new boolean[n + 1];
    int[] prev = new int[n + 1];
    for (int i = 1; i <= n; i++) {
        dist[i] = INF;
        used[i] = false;
        prev[i] = -1;
    }
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingDouble(a -> a[1]));
    pq.add(new int[] {src, 0});
    while (!pq.isEmpty()) {
        int u = pq.poll()[0];
        if (used[u]) continue;
        used[u] = true;
        if (u == dest) break;
        for (int v = 1; v <= n; v++) {
            if (E[u][v] != 0) {
                double alt = dist[u] + E[u][v];
                if (alt < dist[v]) {
                    dist[v] = alt;
                    prev[v] = u;
                    pq.add(new int[] {v, (int)alt});
                }
            }
        }
    }
    if (prev[dest] == -1 && src != dest) {
        if (src == dest) return Collections.singletonList(src);
        return Collections.emptyList();
    }
    List<Integer> path = new ArrayList<>();
    for (int at = dest; at != -1; at = prev[at]) path.add(at);
    Collections.reverse(path);
    return path;
}
```

5. Interactive Features

The application also supports:

- Vertex and edge display
- Name-based search
- Listing by category
- Nearest neighbor lookup
- Shortest path computation with directions
- Interactive menu